



《算法设计与分析》实验报告

姓名 孟 媛 学号 22920202200773

院系 信息学院人工智能系

年级 2020 级 班级 1 班

小组实验任务分工 独立完成

实验时间 2022 年 05 月 1 日

实验名称 《LIS 最长上升子序列》问题的算法分析

第一部分 实验内容

1. 实验目标

- (1) 熟悉使用 LIS 最长上升子序列算法求解问题的基本思路。
- (2) 掌握 LIS 最长上升子序列算法的程序实现方法。
- (3) 理解 LIS 最长上升子序列算法的特点。
- (4) 对 LIS 最长上升子序列问题算法逐步优化并分析复杂度。

2. 实验任务

- (1) 从本学期课堂上分析的算法实例中选择一题，使用多种不同的方法求解之。
- (2) 用文字来描述算法思路，包括解空间、限界函数、算法主要步骤等（不限于以上方面）。
- (3) 在 Windows 环境下使用 C/C++ 语言编程实现算法。
- (4) 记录运行结果，包括输入数据，问题解答及运行时间。
- (5) 分析算法最坏情况下时间复杂度和空间复杂度。
- (6) 谈谈实验后的感想，包括关于该问题或类似问题的求解算法的建议。

3. 实验设备及环境

PC；C/C++ 等编程语言。

4. 实验主要步骤

- (1) 根据实验目标，明确实验的具体任务；
- (2) 设计求解问题所用的算法，并编写程序实现算法；
- (3) 设计实验数据并运行程序、记录运行的结果；
- (4) 分析算法时空性能；
- (5) 实验后的心得体会。

第二部分 问题及算法

• 问题描述

前置概念（子串与子序列）：字符子串指的是字符串中连续的 n 个字符，如 abcdefg 中，ab, cde, fg 等都属于它的子串；字符子序列指的是字符串中不一定连续但先后顺序一致的 n 个字符，即可以去掉字符串中的部分字符，但不可改变其前后顺序。如 abcdefg 中，acd, bdf 属于它的子序列，而 bac, dbfg 则不是，因为它们与字符串的字符顺序不一致。

最长上升子序列 (Longest Increasing Subsequence, 简称 LIS)：假设我们有一个序列 s ，长度为 n ，当 $s[0] < s[1] < s[2] < \dots < s[n-1]$ 的时候，我们称这个序列是上升的。对于给定的一个序列 A 的所有子序列，我们可以从中得到一些上升的子序列 ($S[i_1], S[i_2], \dots, S[i_k]$)，这里 $1 \leq i_1 < i_2 < \dots < i_k \leq N$ 。其中，我们将这些上升子序列中长度最长的一个序列，成为最长上升子序列，即 k 值达到最大。

最长上升子序列 LIS 问题：输入给定一个字符串 S ，输出其最长上升子序列的长度。

1. 方法 1：朴素暴力搜索算法

• 算法思想：

对于给定字符串 S ，解空间设置为它的所有子序列，而对于获取所有子序列这个问题，对于每一个字符，我们可以考虑将其加入和不加入两种情况，如此，便可得到所有子序列。对于每一子序列，判断当前序列是否符合上升规则，符合规则且比答案要长则更新答案，如果不符合则跳过。直至遍历完所有可能的序列。

• 伪码描述：

```

LIS(S)
输入：字符串S
输出：S的最长上升子序列s及其长度l
1. Ans[] <- "", ans <- 0
2. len <- length(S) //获取字符串长度
3. dfs(x, temps, cnt) //x:当前下标,temps:当前字符串,cnt:当前字符串长度
4.     if x > len
5.         return
6.     if cnt == 0
7.         dfs(x + 1, S, temps+S[x], cnt + 1)
8.         dfs(x + 1, S, temps, cnt)
9.         return
10.    if x == len //遍历结束时更新答案
11.        if cnt > ans //若此次上升字符串更长则更新
12.            then ans <- cnt, Ans[] <- temps[0,...,cnt-1]
13.        return
14.    if S[x] > temps[cnt - 1] //我们约定下标从0开始,当前字符加入后符合规则才可以考虑加入的情况
15.        dfs(x + 1, S, temps+S[x], cnt + 1) //
16.    dfs(x + 1, S, temps, cnt) //无论当前字符是否符合上升规则都可以选择不加入
17.    return
18. return Ans[], ans //返回最长上升子序列及其长度

```

- 复杂度分析：

无论给定序列如何，在长度为 n 的情况下，均需要考虑 2^n 种情况，最好、最坏、平均时间复杂度均为 $O(2^n)$ ；空间复杂度也只与 n 有关，为 $O(n)$ 。

此种方法效率过于低下，极其容易爆栈。故不再列入本次实验的测试。

2. 方法 2：简单动态规划算法

- 算法思想：

要求 n 个数的最长上升子序列，可以求前 $n-1$ 个数的最长上升子序列，再判断第 n 个数是否符合规则。求前 $n-1$ 个数的最长上升子序列，可以通过求前 $n-2$ 个数的最长上升子序列……直到求前 1 个数的最长上升子序列，此时 LIS 当然为 1。假设 $d(i)$ 是找以 $S[i]$ 结尾的，在 $S[i]$ 之前的最长上升子序列+1，即前 i 个数的 LIS 长度 + 1。当 $S[i]$ 之前没有比 $S[i]$ 更小的数时， $d(i) = 1$ 。所有的 $d(i)$ 里面最大的那个就是最长上升子序列。

- 状态设计：

$F[i]$ 代表以 $S[i]$ 结尾的 LIS 的长度

- 状态转移：

$$F[i] = \max \{ F[j] + 1, F[i] \} \quad (1 \leq j < i, S[j] < S[i])$$

- 边界处理：

$$F[i] = 1 \quad (1 \leq i \leq n)$$

- 复杂度分析：

对于第 i 个数，需要考虑它前面的最长上升子序列，即遍历 $d[1]$ 到 $d[i-1]$ 寻找最大值。

如此，需要查找的次数便是 $0+1+2+3+\cdots+n-1$ ，时间复杂度为 $O(n^2)$ 。空间上，使用的 F 数组与 S 的长度相同，其他空间大小均为常量，故空间复杂度为 $O(n)$ 。

• 代码实现：

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int maxn = 1e5, INF = 0x7f7f7f7f;
5  int s[maxn], d[maxn];
6  int n, ans = -INF;
7  int main()
8  {
9      cin >> n;
10     for (int i = 1; i <= n; i++)
11     {
12         cin >> s[i];
13         d[i] = 1;
14     }
15     for (int i = 1; i <= n; i++)
16         for (int j = 1; j < i; j++)
17             if (s[j] < s[i]) //如果满足上升条件
18                 d[i] = max(d[i], d[j] + 1); //更新d数组
19     for (int i = 1; i <= n; i++)
20         ans = max(ans, d[i]); //更新ans
21     cout << ans;
22     return 0;
23 }
```

3. 方法 3：二分 + 贪心算法

• 算法思想：

新建一个 low 数组，low[i] 表示长度为 i 的 LIS 结尾元素的最小值。对于一个上升子序列，显然其结尾元素越小，越有利于在后面接其他的元素，也就越可能变得更长。因此，我们只需要维护 low 数组，对于每一个 $S[i]$ ，如果 $S[i] > \text{low}[\text{当前最长的 LIS 长度}]$ ，就把 $S[i]$ 接到当前最长的 LIS 后面，即 $\text{low}[\text{++当前最长的 LIS 长度}] = S[i]$ 。

那么，怎么维护 low 数组呢？对于每一个 $S[i]$ ，如果 $S[i]$ 能接到 LIS 后面，就接上去；否则，就用 $S[i]$ 去更新 low 数组。具体方法是，在 low 数组中找到第一个大于等于 $S[i]$ 的元素 $\text{low}[j]$ ，用 $S[i]$ 去更新 $\text{low}[j]$ 。如果从头到尾扫一遍 low 数组的话，时间复杂度仍是 $O(n^2)$ 。我们注意到 low 数组内部一定是单调不降的，所有我们可以二分 low 数组，找出第一个大于等于 $S[i]$ 的元素。

• 复杂度分析：

二分一次 low 数组的时间复杂度的 $O(\log n)$ ，所以总的时间复杂度是 $O(n \log n)$ 。空间复杂度仍然是 $O(n)$ 。

- 代码实现:

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int maxn = 1e5, INF = 0x7f7f;
5  int low[maxn], s[maxn];
6  int n, ans;
7  int binary_search(int *a, int R, int x) {
8      //二分查找, 返回s数组中第一个>=x的位置
9      int L = 1, mid;
10     while (L <= R) {
11         mid = (L + R) >> 1;
12         if (s[mid] < x)
13             L = mid + 1;
14         else
15             R = mid - 1;
16     }
17     return L;
18 }
19 int main() {
20     scanf("%d", &n);
21     for (int i = 1; i <= n; i++) {
22         scanf("%d", &s[i]);
23         low[i] = INF; //由于low中存的是最小值, 所以low初始化为INF
24     }
25     low[1] = s[1];
26     ans = 1; //初始时LIS长度为1
27     for (int i = 2; i <= n; i++) {
28         if (s[i] > low[ans]) //若s[i]>low[ans], 直接把s[i]接到后面
29             low[++ans] = s[i];
30         else //否则, 找到low中第一个>=s[i]的位置low[j], 用s[i]更新low[j]
31             low[binary_search(low, ans, s[i])] = s[i];
32     }
33     cout << ans; //输出答案
34     return 0;
35 }
```

4. 方法 4: 贪心（在 3 的基础上） + lower_bound 优化

- 优化思想:

由于长度相同的上升子序列只需要保存结尾最小的那个, 而长度递增时, 结尾数字的大小也是递增的。最长上升子序列就是找出比他大的第一个数。前面的数都比他小, 所以他和这个数的长度相同。然后由于他比较然后小, 更新找到的那个值。

- 优化后的代码（时空复杂度不变）:

```

1  #include <iostream>
2  #include <algorithm>
3  #include <cstring>
4  using namespace std;
5  const int maxn = 1e5, INF = 0x7f7f7f7f;
6  int low[maxn], d[maxn], n, s[maxn];
7  int main() {
8      cin >> n;
9      for (int i = 0; i < n; i++) {
10         cin >> s[i];
11         low[i] = INF;
12     }
13     int ans = -1;
14     for (int i = 0; i < n; i++) {
15         int j = lower_bound(low, low + n, s[i]) - low;
16         d[i] = j + 1;
17         if (ans < d[i])
18             ans = d[i];
19         low[j] = s[i];
20     }
21     cout << ans;
22     return 0;
23 }

```

5. 方法 5: 动态规划（在 2 的基础上） + 树状数组维护

- 优化思想:

$$F[i] = \max \{ F[j] + 1, F[i] \} \quad (1 \leq j < i, A[j] < S[i])$$

我们在递推 F 数组的时候，每次都要把 F 数组扫一遍求 $F[j]$ 的最大值，时间开销比较大。我们可以借助数据结构来优化这个过程。用树状数组来维护 F 数组（据说分块也是可以的，但是分块是 $O(n \cdot \sqrt{n})$ 的时间复杂度，不如树状数组跑得快），首先把 A 数组从小到大排序，同时把 $S[i]$ 在排序之前的序号记录下来。然后从小到大枚举 $S[i]$ ，每次用编号小于等于 $S[i]$ 编号的元素的 LIS 长度+1 来更新答案，同时把编号大于等于 $S[i]$ 编号元素的 LIS 长度+1。因为 A 数组已经是有序的，所以可以直接更新。

- 优化后的代码：时空复杂度不变

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int maxn = 1e5;
4  int a[maxn], b[maxn], n, sz, ans;
5  int dp[maxn], f[maxn];
6  int lowbit(int x) {
7      return x & (-x);
8  }
9  void motify(int x, int w) {
10     for (; x <= sz; x += lowbit(x)) {
11         f[x] = max(f[x], w);
12     }
13 }
14 int get(int x) {
15     int temp = 0;
16     for (; x; x -= lowbit(x)) {
17         temp = max(temp, f[x]);
18     }
19     return temp;
20 }
21 int main() {
22     cin >> n;
23     for (int i = 1; i <= n; i++) {
24         scanf("%d", &a[i]);
25         b[i] = a[i];
26     }
27     sort(b + 1, b + n + 1);
28     sz = unique(b + 1, b + n + 1) - (b + 1);
29     for (int i = 1; i <= n; i++) {
30         a[i] = lower_bound(b + 1, b + sz + 1, a[i]) - b;
31     }
32     ans = 0;
33     for (int i = 1; i <= n; i++) {
34         dp[i] = get(a[i] - 1) + 1; //最长上升
35         // dp[i]=get(a[i])+1; //最长不下降
36         ans = max(ans, dp[i]);
37         motify(a[i], dp[i]);
38     }
39     cout << ans;
40     return 0;
41 }

```


第三部分 实验结果与分析

1. 实验数据及结果

- 当数据规模为 100

Local: 2	Local: 3	Local: 4	Local: 5
^ Testcase 1 Passed 52ms Input: Copy 100 2264788 115448 2032319 7535338 2898066 1041732 757891 6822971 5090063 4393852 5188806 5219292 4881557 9302946 Expected Output: Copy 20 Received Output: Copy 20	^ Testcase 1 Passed 55ms Input: Copy 100 2264788 115448 2032319 7535338 2898066 1041732 757891 6822971 5090063 4393852 5188806 5219292 4881557 9302946 Expected Output: Copy 20 Received Output: Copy 20	^ Testcase 1 Passed 51ms Input: Copy 100 2264788 115448 2032319 7535338 2898066 1041732 757891 6822971 5090063 4393852 5188806 5219292 4881557 9302946 Expected Output: Copy 20 Received Output: Copy 20	^ Testcase 1 Passed 55ms Input: Copy 100 2264788 115448 2032319 7535338 2898066 1041732 757891 6822971 5090063 4393852 5188806 5219292 4881557 9302946 Expected Output: Copy 20 Received Output: Copy 20

- 当数据规模为 10000

Local: 2	Local: 3	Local: 4	Local: 5
^ Testcase 1 Passed 342ms Input: Copy 10000 8558367 6738908 213518 5687454 5898053 1116263 1509020 5345203 6157251 6745356 2227399 2740901 9390483 272124 Expected Output: Copy 197 Received Output: Copy 197	^ Testcase 1 Passed 62ms Input: Copy 10000 8558367 6738908 213518 5687454 5898053 1116263 1509020 5345203 6157251 6745356 2227399 2740901 9390483 272124 Expected Output: Copy 197 Received Output: Copy 197	^ Testcase 1 Passed 66ms Input: Copy 10000 8558367 6738908 213518 5687454 5898053 1116263 1509020 5345203 6157251 6745356 2227399 2740901 9390483 272124 Expected Output: Copy 197 Received Output: Copy 197	^ Testcase 1 Passed 63ms Input: Copy 10000 8558367 6738908 213518 5687454 5898053 1116263 1509020 5345203 6157251 6745356 2227399 2740901 9390483 272124 Expected Output: Copy 197 Received Output: Copy 197

- 当数据规模为 100000（防止越界取 99999）

Local: 2	Local: 3	Local: 4	Local: 5
^ Testcase 1 Failed Timed Out Input: Copy 99999 6031414 5008874 7536176 593212 3502311 7685987 8960859 145297 7945112 2596566 6614973 4310415 2727921 2073825 5730435 Expected Output: Copy Received Output: Copy SIGTERM	^ Testcase 1 Passed 114ms Input: Copy 99999 6031414 5008874 7536176 593212 3502311 7685987 8960859 145297 7945112 2596566 6614973 4310415 2727921 2073825 5730435 Expected Output: Copy 612 Received Output: Copy 612	^ Testcase 1 Passed 163ms Input: Copy 99999 6031414 5008874 7536176 593212 3502311 7685987 8960859 145297 7945112 2596566 6614973 4310415 2727921 2073825 5730435 Expected Output: Copy 612 Received Output: Copy 612	^ Testcase 1 Passed 162ms Input: Copy 99999 6031414 5008874 7536176 593212 3502311 7685987 8960859 145297 7945112 2596566 6614973 4310415 2727921 2073825 5730435 Expected Output: Copy 612 Received Output: Copy 612

2. 实验分析及结论

通过上述实验结果，我们不难看出，当数据规模较小时，四者这种时间上的差异并不明显。但当数据规模逐步扩大时，方法 3/4/5 有着 $O(n \log n)$ 时间复杂度的 LIS 算法要比有着 $O(n^2)$ 时间复杂度的方法 2 优秀的多。尤其是在处理大规模数据时，方法 2 已经运行超时，而后三者的时间仍较为优秀。

第四部分 心得与展望

1. 自我评价及心得体会

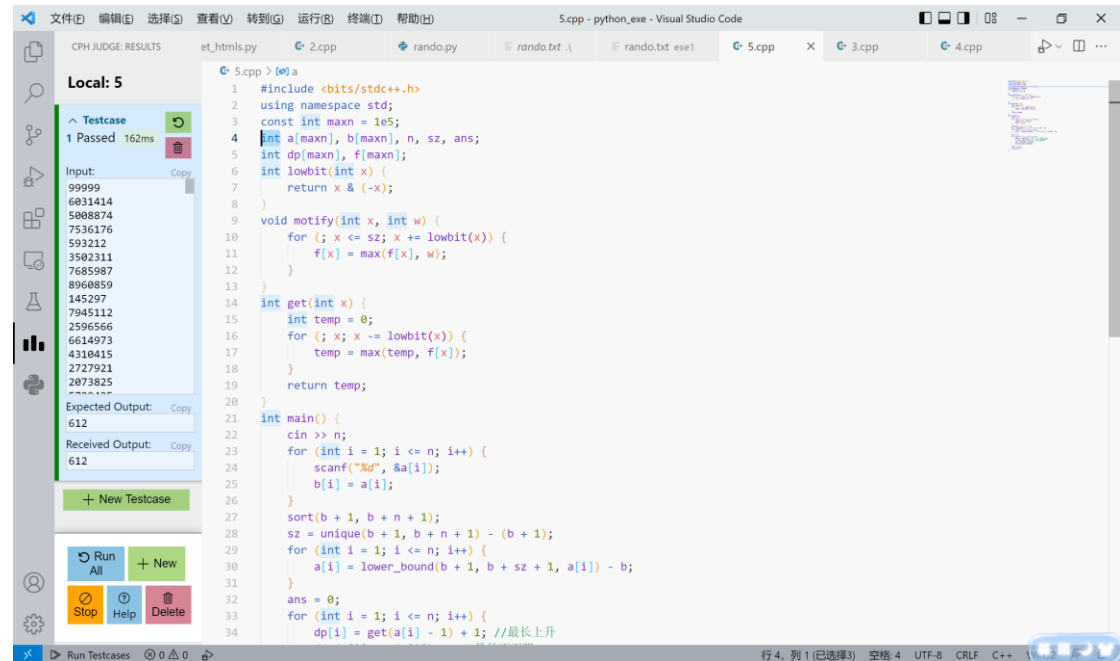
本次实验，我进一步加深了对 LIS 问题的理解，并且尝试学习了不同有优化方式对问题的改进，总体来说收获颇丰。从前对时间复杂度的概念总觉得有些虚无缥缈，但今天切切实实地感受到了当数据规模过大时产生的天差地别的差异。

2. 展望

希望在以后的算法问题研究中，也能循序渐进，逐步体会对同一问题的不同优化算法，从而更好地理解时空复杂度对于解决问题效率的影响。

第五部分 附录

1. 程序主要界面



2. 源程序

前面已贴。

参考文献

https://blog.csdn.net/lxt_Lucia/article/details/81206439